

06 - Saturazione

Si introduce la tecnica della **saturazione**, che permette la risoluzione di svariati problemi in un ambiente distribuito con una topologia ad albero.

Tecnica della saturazione

Per applicare questa tecnica, si consideri una rete distribuita con una topologia ad albero e sia tale informazione, riguardante la struttura topologica, nota a tutti i nodi della rete. In particolar modo, ogni nodo sa se è una foglia oppure un nodo interno. La tecnica della saturazione consiste nell'identificare una singola coppia di nodi adiacenti all'interno della rete. Tale coppia di nodi sarà in grado di far iniziare altre computazioni, a seconda del problema che si vuole risolvere. In altri termini, la tecnica della saturazione permette di eleggere un arco della rete: questa tecnica prende anche il nome di *link election*. Si osserva che non c'è nessuna garanzia su quale arco venga eletto: questa tecnica garantisce l'elezione di uno tra i possibili archi, ma non garantisce che sia proprio uno specifico arco ad essere eletto.

Tipicamente la tecnica della saturazione è usata per risolvere problemi distribuiti di varia natura. In particolare, la saturazione è inserita all'interno di un processo più grande, che può essere diviso nelle seguenti tre fasi:

- **Fase di Attivazione**, nella quale i nodi iniziatori attivano i restanti nodi della rete utilizzando un protocollo di WAKE-UP;
- **Fase di Saturazione**, nella quale a partire dalle foglie si identifica un'unica coppia di nodi adiacenti, anche detti *nodi saturati*;
- **Fase di Risoluzione**, nella quale la coppia di nodi saturati danno inizio ad una o più computazioni necessarie alla risoluzione di uno specifico problema di interesse.

Essendo che la natura della terza fase dipende dall'applicazione, si formalizzano le prime due fasi del processo appena descritto, definendo la tripla $P = \langle P_{init}, P_{final}, R \rangle$ dove, sia V l'insieme delle entità che costituisce la rete e $W \subseteq V$ tale che $W \neq \emptyset$ l'insieme degli initiator,

- $P_{init} = \forall x \in V, \text{status}(x) = \begin{cases} \text{awake} & x \in W \\ \text{asleep} & x \in V \setminus W \end{cases}$
- $P_{final} : \exists x, y \in V : [(x, y) \in E \wedge \text{status}(x) = \text{status}(y) = \text{saturated}] \wedge \forall z \in V, z \neq y, z \neq x : [\text{status}(z) = \text{active}]$, ossia, tutti i nodi tranne due sono ACTIVE, i due nodi restanti sono SATURATED e adiacenti.
- $R_{saturation} = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \\ \text{Knowledge of the Topology (KT)} \\ \text{Ordered Messages (MO)} \end{cases}$

Dove P_{init} è un predicato che definisce l'insieme di possibili configurazioni iniziali ammissibili nella fase di attivazione, mentre P_{final} è un predicato sull'insieme di possibili configurazioni nel quale il sistema si deve trovare al termine della fase di saturazione.

Protocollo FULL-SATURATION

Il protocollo fa uso degli stati seguenti

- $S = \{\text{available, active, processing, saturated}\}$
- $S_{init} = \{\text{available}\}$; Inizialmente, tutti i nodi si trovano nello stato available. Tale stato rappresenta come un nodo sia disponibile ad iniziare l'esecuzione di una computazione. Non è detto però che un nodo available sia effettivamente sveglio (ossia in uno stato di attivazione): se un nodo nello stato available riceve un impulso spontaneo dall'esterno, allora questo si sveglia, entrando nello stato active, e invia un messaggio di wake-up ai suoi vicini, svolgendo il ruolo di initiator. Se un agente in stato available riceve un messaggio di wake-up da suo vicino, allora passa nello stato active ed invia un messaggio di wake-up a tutti i suoi vicini eccetto il sender. In particolar modo, quindi, ad alto livello, tra i nodi available possono distinguersi i nodi awake ed i nodi asleep nella configurazione iniziale.

Idealmente, per la definizione del problema, si vuole che l'insieme di stati terminali sia $S_{term} = \{\text{processing, saturated}\}$, ma, come detto in precedenza, la tecnica della saturazione è una componente di un processo suddiviso in tre fasi, delle quali si analizzano ora le prime due, essendo l'ultima dipendente dall'applicazione specifica. Di conseguenza si definisce un protocollo "plug-in", ossia tale per cui, nella sua esecuzione, non tutte le entità entreranno prima o poi in uno stato terminale. Per trasformare il protocollo FULL-SATURATION in un protocollo completo, devono essere eseguite altre azioni (ossia quelle della fase di risoluzione) in maniera tale che, in un tempo finito, tutte le entità entrino in uno stato terminale. Di conseguenza si indica con $S_{final} = \{\text{processing, saturated}\}$ l'insieme di stati finali, ossia quegli stati in cui si trovano gli agenti al completamento del task distribuito.

La fase di attivazione è una fase di wake-up: ogni iniziatore invia un messaggio di attivazione a tutti i suoi vicini ed entra nello stato active; Ogni nodo non iniziatore, alla ricezione di un messaggio di attivazione da parte di un vicino, entra nello stato active ed invia un messaggio di wake-up ad ogni suo altro vicino. I nodi attivi ignorano ogni altro messaggio di attivazione.

In tempo finito, tutti i nodi diventano attivi, incluse le foglie. Saranno proprio quest'ultime ad iniziare la seconda fase della computazione.

Ogni foglia attiva inizia la fase di saturazione inviando un messaggio M al proprio unico vicino, al quale ci si riferisce come "padre" della foglia, ed entra nello stato di processing. Si osserva esplicitamente che, per la definizione delle restrizioni, in tempo finito i messaggi M giungeranno ai nodi interni.

Un nodo interno attende fino a quando riceve i messaggi M da tutti i suoi vicini eccetto uno, invia poi M a tale nodo, il quale diventerà suo "padre", ed entra poi nello stato di processing.

Se un nodo x nello stato di processing riceve un messaggio M proveniente dal suo nodo padre, allora tale nodo x diventa saturato.

Note

Il task distribuito può essere completato anche se il protocollo non termina.
Si guardi il primo protocollo per la risoluzione del broadcasting.

Il comportamento dei nodi è descritto dalle seguenti regole

```
if state == "AVAILABLE":
    spontaneously:
        send("ACTIVATE") to N(x)
        INITIALIZE
        Neighbors := N(x)
        if |Neighbors| = 1:
            PREPARE_MESSAGE
            parent = extract(Neighbors)
            send(M) to parent
            state = "PROCESSING"
        else:
            state = "ACTIVE"

    receiving("ACTIVATE"):
        send("ACTIVATE") to N(x) - {sender}
        INITIALIZE
        Neighbors := N(x)
        if |Neighbors| = 1:
            PREPARE_MESSAGE
            parent = extract(Neighbors)
            send(M) to parent
            state = "PROCESSING"
        else:
            state = "ACTIVE"

if state == "ACTIVE":
    receiving(M):
        PROCESS_MESSAGE
        Neighbors = Neighbors - {sender}
        if |Neighbors| = 1:
            PREPARE_MESSAGE
            parent = extract(Neighbors)
            send(M) to parent
            state = "PROCESSING"

if state == "PROCESSING":
```

```

receiving(M):
    PROCESS_MESSAGE
    RESOLVE

if state == "SATURATED":
    None

```

Si osserva che nel protocollo il concetto di *parent* è utilizzato in due modi differenti

- Nello stato available, si intende il padre di un nodo foglia, ovvero l'unico nodo adiacente al nodo foglia;
- Nello stato active, il parent di un nodo x rappresenta l'unico nodo adiacente che ancora non ha inviato il messaggio M al nodo x .

Si definiscono le procedure utilizzate nel protocollo

```

Procedure INITIALIZE:
    None

Procedure PREPARE_MESSAGE:
    M := ("SATURATION")

Procedure PROCESS_MESSAGE:
    None

Procedure RESOLVE:
    status = "SATURATED"
    start Resolution stage

```

Questi metodi possono venir "sovrascritti" a seconda dello scenario applicativo.

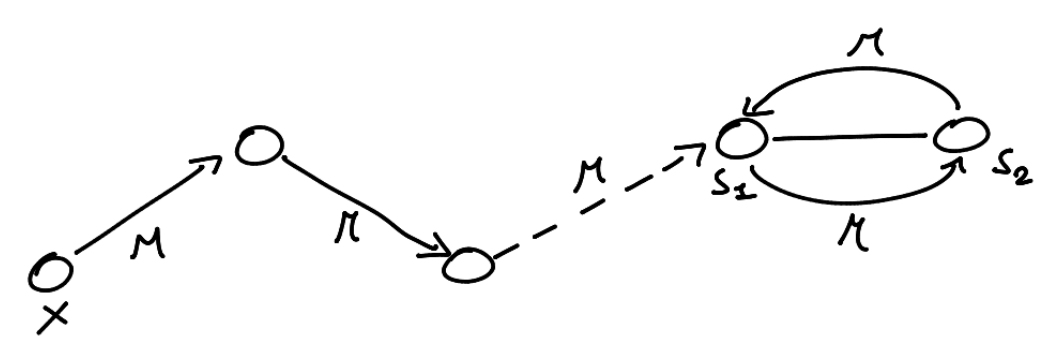
Correttezza

Per mostrare la correttezza del protocollo, si deve argomentare il fatto che esattamente due nodi passano dallo stato processing allo stato saturated, e che questi due nodi sono adiacenti.

Si osserva che

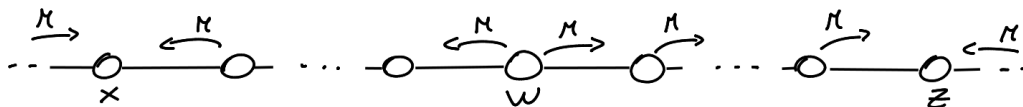
- Un nodo invia un messaggio M solo al suo parent.
- Un nodo passa nello stato di processing solo dopo che ha mandato il messaggio M al suo parent;
- Un nodo passa nello stato saturated solo dopo che, trovandosi nello stato di processing, riceve il messaggio M dal suo parent.

Sia quindi x un nodo arbitrario e si consideri il percorso fatto dai messaggi M a partire da x



Dato che non ci sono cicli, ad un certo punto del cammino si incontra un nodo s_1 che si trova nello stato saturated, in quanto ha ricevuto dal suo parent, il nodo s_2 , il messaggio M . Essendo che s_2 ha inviato un messaggio M ad s_1 , allora s_2 si è dovuto trovare necessariamente nello stato processing e deve aver considerato s_1 come suo parent. Quindi, quando il messaggio M spedito da s_1 arriva ad s_2 , anche quest'ultimo agente entra nello stato saturated. Dunque il protocollo trova sempre almeno due nodi adiacenti saturati.

Si dimostra ora che i nodi saturati sono esattamente due. Si assuma per assurdo che il protocollo faccia sì che tre i nodi entrino nello stato saturated, siano questi x, y, z . Data la topologia ad albero, due di questi nodi non sono adiacenti. Si consideri il cammino tra questi nodi non adiacenti, siano essi x e z .



Allora per definizione del protocollo, x e z diventano saturati quando ricevono il messaggio M . Se i due nodi non sono adiacenti, deve esistere un nodo w nel cammino che invia due messaggi M : uno verso z e uno verso x . Ma per definizione del protocollo, w non può inviare M a due archi.

In conclusione, si osserva che in ogni caso la scelta di quali nodi vengono saturati dal protocollo dipende dal ritardo dei messaggio. Questo implica che potenzialmente ogni coppia di nodi adiacenti può essere la coppia saturata.

Complessità

Per quanto riguarda la **message complexity**, vale che:

- Nella fase di **attivazione**, vale la complessità del protocollo WAKE-UP e dato che in un albero $m = n - 1$, allora vengono spediti $n + k - 2$ messaggi, dove k indica il numero di iniziatori;
 - Nella fase di **saturazione** vengono inviati n messaggi;
- Allora in totale la message complexity del protocollo è pari a

$$M[\text{Saturation}] = 2n + k - 2$$

Per quanto riguarda la **time complexity**, si ricorda che valgono le solite ipotesi di synchronized clocks e unitary communication delay. Sia $I \subseteq V$ l'insieme degli iniziatori e $L \subseteq V$ l'insieme delle foglie. Sia $t(x)$ il ritardo temporale necessario affinché, dall'istante $t = 0$ in cui inizia l'esecuzione dell'algoritmo, il nodo x entri nello stato *attivo*. Per diventare saturo, il nodo s deve aver aspettato sino a quando tutte le foglie sono diventate *attive*, e i messaggi M provenienti da esse lo hanno raggiunto, ossia, deve aver aspettato $\max\{t(l) + d(l, s) : l \in L\}$.

Un nodo non-iniziatore x , per esser diventato attivo, deve aver aspettato che un messaggio di attivazione lo abbia raggiunto. Un iniziatore, invece, non richiede tempo di attesa aggiuntivo, quindi $t(x) = \min\{d(x, y) + t(y) : y \in I\}$.

Il ritardo totale dall'inizio dell'esecuzione del protocollo, sino a quando s diventa saturo, è quindi

$$T[\text{Saturation}] = \max\{\min\{d(l, y) + t(y)\} + d(l, y) : y \in I, l \in L\}$$

Applicazioni della saturazione

La tecnica della saturazione può essere utilizzata per risolvere vari problemi computazionali in modo distribuito.

Si studiano quindi diversi task che utilizzano la tecnica della saturazione. In particolar modo, si studiano le fasi di risoluzione dei problemi presi in analisi, precedute dalle due fasi descritte in precedenza.

Calcolo del minimo

Si consideri il task distribuito in cui ogni nodo x detiene un valore $value(x)$, e al termine del task ogni nodo x deve sapere se $value(x)$ è il minimo oppure no rispetto ai valori detenuti dagli altri nodi, entrando quindi in uno stato appropriato, siano questi rispettivamente minimum e large. I valori non sono necessariamente distinti, dunque possono esistere più nodi che detengono il minimo valore. Tutti gli agenti aventi tale valore minimo devono quindi entrare nel corrispettivo stato.

Si osserva che in un albero radicato il problema è banale. In tale ambiente è sufficiente che ogni nodo invii il proprio valore alla radice, che calcola il minimo e poi lo distribuisce nella rete, in modo da far capire ad ogni nodo se il proprio valore è il minimo oppure no.

Questo non è possibile in albero generico, non radicato. In questo caso, si applica la tecnica della saturazione:

- I nodi foglia inviano il messaggio di saturazione M al proprio padre, allegando il proprio valore;
- Quando un nodo interno riceve tutti i messaggi di saturazione M contenenti i valori di ogni nodo figlio, ne calcola il minimo, comprendendo anche il suo valore, e lo invia tramite messaggio M al nodo genitore determinato dalla procedura di saturation;

- I due leader eletti al termine del processo di saturazione conoscono il valore del minimo, dunque lo inviano in broadcast agli altri nodi.

Il protocollo è il seguente, dove le coppie (*stato*, *evento*) e le funzioni mancanti sono uguali al protocollo plug-in originale, mentre le nuove coppie e le nuove funzioni esplicitate sovrascrivono le precedenti.

- $S = \{\text{available, active, processing, saturated, minimum, large}\}$
- $S_{init} = \{\text{available}\}$
- $S_{term} = \{\text{minimum, large}\}$

```

if state == "PROCESSING":
    receiving(Notification):
        send(Notification) to N(x) - {parent}
        if value(x) = Received_Value:
            state = "MINIMUM"
        else:
            state = "LARGE"

Procedure INITIALIZE:
    min = value(x)

Procedure PREPARE_MESSAGE:
    M = {"SATURATION", min}

Procedure PROCESS_MESSAGE:
    min = MIN(min, Received_Value) //MIN funzione che restituisce il
    minimo

Procedure RESOLVE:
    Notification = {"RESOLUTION", min}
    send(Notification) to N(x) - {parent}
    if value(x) == min:
        state == "MINIMUM"
    else:
        state == "LARGE"

```

Quindi, ogni agente include nel messaggio M il più piccolo valore che conosce al padre. In particolar modo, nella fase di saturazione le foglie invieranno il loro valore con il messaggio M , ed ogni nodo interno invia al proprio padre il più piccolo valore tra il suo e quelli ricevuti. La fase di risoluzione consiste semplicemente nel broadcasting, che parte dai nodi saturi, del minimo valore che hanno calcolato.

Complessità

Per quanto riguarda la **message complexity**, vengono inviati $2n + k - 2$ messaggi per la saturazione e $n - 2$ messaggi per il broadcast finale, essendo che quest'ultima procedura coinvolge tutti gli archi della rete eccetto quello eletto, dunque in totale si ottiene

$$3n + k - 4$$

dove k indica il numero di iniziatori.

L' **ideal time complexity** è quella di Saturation, più quella richiesta dalla propagazione della notifica nella fase di risoluzione. Sia Sat l'insieme dei due nodi saturati, allora la complessità temporale è pari a

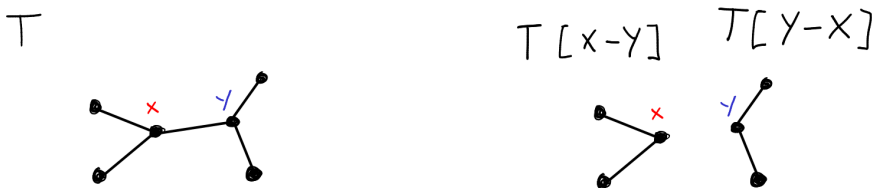
$$T[\text{Saturation}] + \max\{d(s, x) : s \in Sat, x \in V\}$$

Correttezza

La correttezza segue dal fatto che entrambi i nodi saturati conoscono il valore minimo.

Si dimostra formalmente.

Siano $x, y \in V$ due nodi tali per cui $(x, y) \in E$. Essendo che la rete T ha una topologia ad albero, la rimozione dell'arco (x, y) porterà alla definizione di due sottoalberi: Sia $T[x - y]$ il sottoalbero contenente x ma non y e, specularmente, $T[y - x]$ il sottoalbero contenente y ma non x .



Si mostra che se x invia un messaggio M al suo vicino y , allora M contiene il più piccolo valore in $T[x - y]$. Lo si fa per induzione sull'altezza h di $T[x - y]$.

Per $h = 1$, allora è vero essendo x una foglia.

Sia per ipotesi vero per $k \geq 1$.

Si mostra che è vero per $h = k + 1$:

Si supponga che x abbia inviato M ad y . Allora, x ha ricevuto un valore da tutti i suoi altri vicini $y_1, y_2, \dots$, ed essendo che $T[y_i - x]$ ha altezza minore di h per ogni vicino $y_i \neq y$ allora, per ipotesi induttiva, il valore inviato da y_i ad x è il minimo in $T[y_i - x]$. Questo vuol dire che il più piccolo valore tra $value(x)$ e tutti i valori ricevuti da x è il minimo valore in $T[x - y]$, ed è proprio il valore che x invia ad y .

Essendo che un nodo saturo riceve, per definizione del protocollo, un messaggio M da tutti i suoi vicini, allora conosce il valore minimo della rete.

Calcolo di funzioni distribuito

In generale, la saturazione può essere utilizzata per calcolare una classe di funzioni $F(\cdot)$ in modo distribuito su una topologia ad albero. In particolare, sia F un'operazione di semi-gruppo, ovvero che rispetti le seguenti proprietà

- **Associatività:** $F(F(a, b), c) = F(a, F(b, c))$
 - **Commutatività:** $F(a, b) = F(b, a)$
- Operazioni di questo tipo sono ad esempio il calcolo del massimo, del minimo, la somma, il prodotto e i connettori logici.

Il protocollo è il seguente, dove le coppie (*stato, evento*) e le funzioni mancanti sono uguali al protocollo plug-in originale, mentre le nuove coppie e le nuove funzioni esplicitate sovrascrivono le precedenti.

Il protocollo è quindi il protocollo Full Saturation dove le procedure sono state modificate opportunamente, e dove la fase di risoluzione consiste nel broadcasting, iniziato dai nodi saturati, del risultato finale della funzione calcolata.

- $S = \{\text{available, active, processing, saturated, done}\}$
- $S_{init} = \{\text{available}\}$
- $S_{term} = \{\text{done}\}$

```

if state == "PROCESSING":
    receiving(Notification):
        result = received_value
        send(Notification) to N(x) - {parent}
        state = "DONE"

Procedure INITIALIZE:
    if value(x) != NULL:
        result = F(value(x))
    else:
        result = NULL

Procedure PREPARE_MESSAGE:
    M = {"SATURATION", result}

Procedure PROCESS_MESSAGE:
    if received_value != NULL:
        if result != NULL:
            result = F(result, received_value)
        else:
            result = F(received_value)

Procedure RESOLVE:
    Notification = {"RESOLUTION", result}
    send(Notification) to N(x) - {parent}
    state = "DONE"

```

Correttezza e complessità

La correttezza segue dal fatto che entrambi i nodi saturi conoscono il risultato della funzione, ed è analoga alla dimostrazione fatta in precedenza:

Il tempo e la message complexity del protocollo sono esattamente uguali a quelle del minimum finding. Quindi, è possibile eseguire le operazioni di semigrupp in maniera ottimale su un albero con un numero arbitrario di initiators, e senza che l'albero possenga una radice o ulteriori informazioni.

Calcolo della media

Anche il calcolo della media può essere effettuato sempre tramite saturazione. In questo caso i messaggi M sono utilizzati per calcolare sia il numero di nodi, sia la somma del valore dei nodi. Una volta che queste informazioni vengono passate ad uno dei due nodi saturati, si è in grado di calcolare il valore medio tra tutti i valori dei nodi.

In particolar modo, si osserva che $Average \equiv Sum/Size$, dove Sum è la somma di tutti i valori rilevanti, mentre $Size$ è il numero di tali valori. Essendo che Sum è un'operazione di semigrupp, si può calcolare come definito in precedenza. Inoltre, la $Size$ è calcolabile immediatamente usando la saturazione

```
if state == "PROCESSING":
    result = received_value
    send(Notification) to N(x)-{parent}
    state = "DONE"

Procedure INITIALIZE:
    sum = value(x)
    size = 1

Procedure PREPARE_MESSAGE:
    M = ("SATURATION", sum, size)

Procedure PROCESS_MESSAGE:
    sum = sum + received_sum
    size = size + received_size

Procedure RESOLVE:
    result = sum/size
    Notification = ("RESOLUTION", result)
    send (Notification) to N(x)-{parent}
    state = "DONE"
```

Calcolo delle eccentricità

Sia $d(x, y)$ la distanza tra x e y nella rete.

Definizione (Eccentricità di un nodo)

In un grafo $G = (V, E)$ l'**eccentricità** di un nodo $x \in V$ è definita come la massima distanza tra x e un qualsiasi altro nodo $y \in V$. Formalmente

$$r(x) := \max_{y \in V} \{d(x, y)\}$$

Il calcolo di $r(x)$ esprime quanto *centrale* è il nodo x nella rete.

Si analizzano due protocolli che calcolano in modo distribuito l'eccentricità per ogni nodo della rete: il primo non usa la tecnica della saturazione, mentre il secondo la utilizza.

Si osserva esplicitamente che, nelle precedenti applicazioni, si sono risolti problemi a singoli valori, ossia problemi tali per cui la soluzione richiede l'identificazione di un singolo valore.

Nel calcolo delle eccentricità si vuole determinare tale misura *per ogni nodo della rete*.

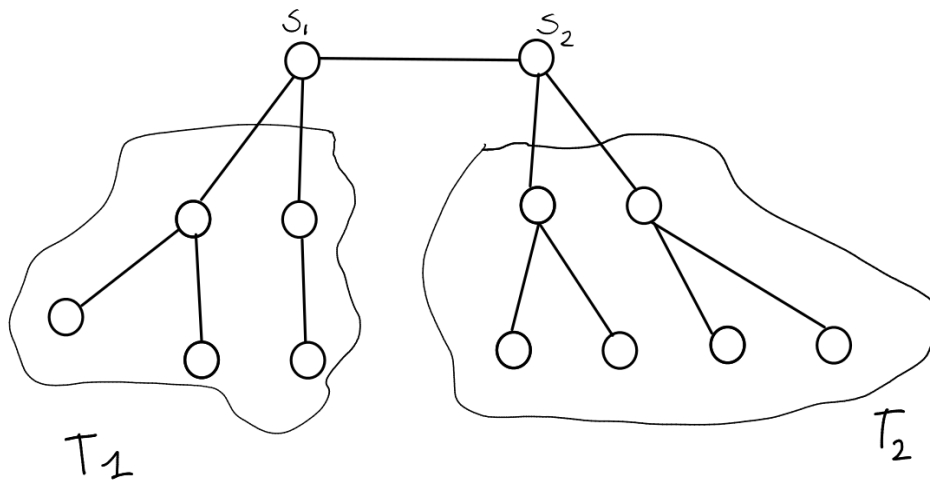
Protocollo 1

La prima idea di un protocollo che calcola $r(x) \forall x$ è la seguente:

1. Il nodo x invia una richiesta in broadcast a tutti i nodi della rete. In questa richiesta viene aggiornato volta per volta il numero di archi attraversati;
2. Le foglie inviano un messaggio indietro a x per comunicare quanto si trovano distanti da x (convergecast).
In questo modo ogni nodo x è in grado di collezionare tutte le distanze, e dunque calcolare qual'è la distanza massima, ovvero la sua eccentricità.
In termini di *message complexity* il calcolo di $r(x)$ richiede $2(n - 1)$ messaggi. Allora il calcolo di tutte le eccentricità richiede $2(n^2 - n)$ messaggi.

Protocollo 2 (con saturazione)

Questo protocollo utilizza la saturazione per eleggere un particolare arco (s_1, s_2) , graficamente

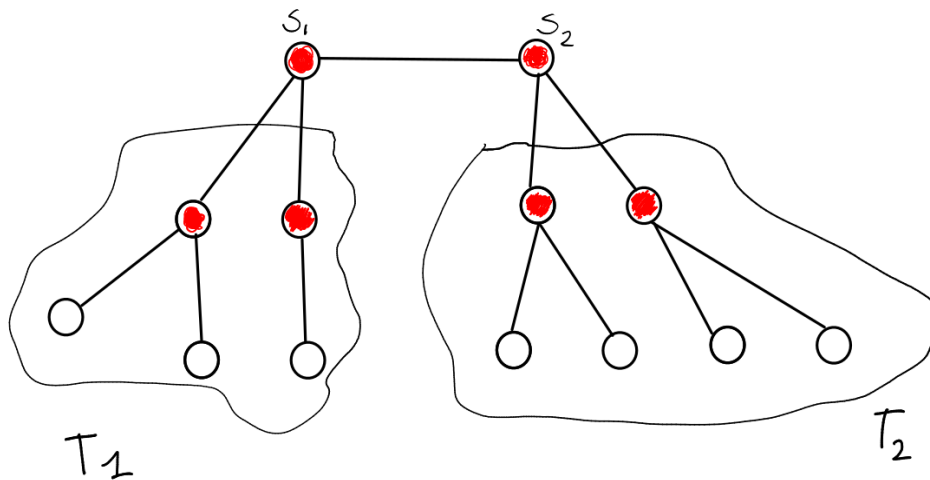


dove h_1, h_2 sono rispettivamente le altezze dei sottoalberi T_1, T_2 , radicati rispettivamente in s_1 ed s_2 . Queste altezze possono essere calcolate direttamente durante il processo di saturazione.

Con tali informazioni, i nodi s_1 ed s_2 sono in grado di calcolare la loro eccentricità. Infatti,

- $r(s_1) = \max\{h_1, h_2 + 1\};$
- $r(s_2) = \max\{h_2, h_1 + 1\};$

Una volta calcolati $r(s_1)$ e $r(s_2)$ i nodi che si trovano a distanza 1 da s_1 e s_2 sono in grado di calcolare a loro volta la loro eccentricità.

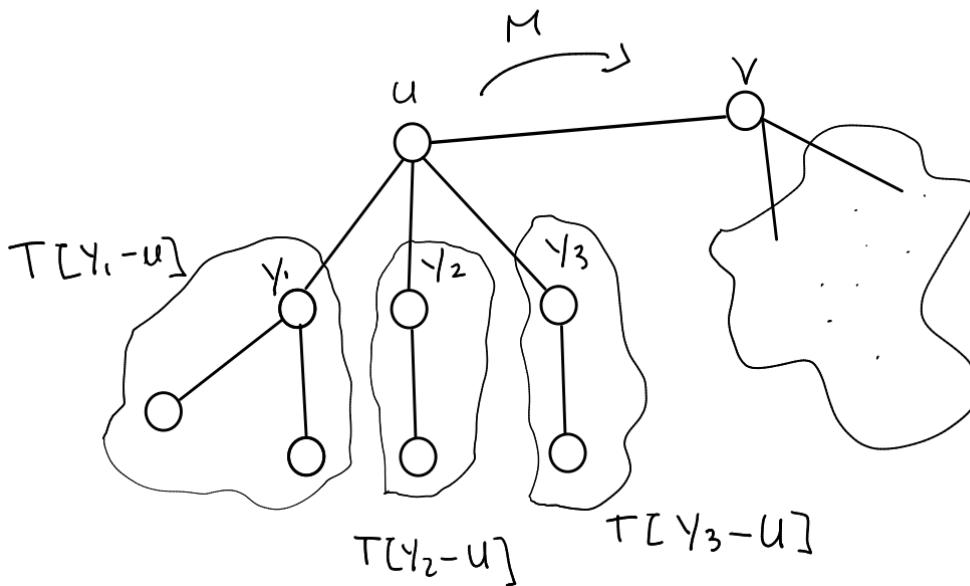


In particolar modo, per far sì che i nodi saturi possano calcolare la loro eccentricità, è sufficiente includere nel messaggio M inviato da un nodo x ad un nodo y la massima distanza tra x ed i nodi in $T[x - y]$ incrementata di uno. In tale maniera, ogni nodo saturo s conoscerà la massima distanza tra ogni suo vicino y e i nodi in $T[y - s]$, e può quindi

determinare la sua eccentricità.

Si vuole che però *tutti* i nodi, e quindi non solo quelli saturi, determinino la propria eccentricità.

Si consideri un'entità u . Questa invia un messaggio M al padre v dopo aver ricevuto un messaggio dello stesso tipo da ogni altro vicino y_i . Per ogni i , il messaggio inviato da y_i ad u contiene la massima distanza tra u e i nodi in $T[y_i - u]$, in formule, $\max\{d(u, z) : z \in T[y_i - u]\}$.



In altre parole, u conosce già tutte la massima distanza tra tutte le entità eccetto quelle contenute nell'albero $T[v - u]$. Quindi, l'unica informazione che manca ad u è $\max\{d(u, y) : y \in T[v - u]\}$.

Si osserva che

$$\max\{d(u, y) : y \in T[v - u]\} = 1 + \max\{d[v, z] : z \in N(v) \setminus \{u\}\}$$

dove $d[v, z] = \max\{d(v, k) : k \in T[z - v]\}$ è la più grande distanza tra v e i nodi in $T[z - v]$.

A parole, la massima distanza tra u ed i nodi in $T[v - u]$ è data dalla massima distanza tra v ed un qualsiasi altro nodo raggiungibile da v nel caso in cui si rimuovesse l'arco (u, v) da T , incrementata di 1.

Sintetizzando, ad ogni nodo, eccetto quelli saturi, manca un'informazione: la massima distanza tra tale nodo e i nodi dall'altro lato del link che lo connette al padre. Se i nodi padre potessero fornire tale informazione ai rispettivi figli, il task può essere portato a termine. Sfortunatamente, i nodi padre a loro volta non posseggono l'informazione, a meno che questi non siano nodi saturi.

I nodi saturi posseggono tutte le informazioni necessarie. Dispongono anche

dell'informazione necessaria ai propri vicini: **sia s un nodo saturo ed x un suo vicino non saturo. Ad x manca l'informazione $\max\{d(x, y) : y \in T[s - x]\}$. Dall'equazione vista in precedenza, tale valore è proprio**

$$\max\{d(x, y) : y \in T[s - x]\} = 1 + \max\{d[s, z] : z \in N(s) \setminus \{x\}\}$$

con $d[s, z] = \max\{d(s, k) : k \in T[z - s]\}$, **ed s conosce tutte le $d[s, z]$, essendo che queste sono state incluse nei messaggi M ricevuti. Quindi, il nodo saturo s può fornire tali informazioni ai suoi vicini, i quali possono poi calcolare la propria eccentricità. La proprietà rilevante affinché tutti i nodi possano calcolare la propria eccentricità, è data dal fatto che, dal momento che i vicini di s conoscono tali informazioni, allora possono condividerle con gli altri loro vicini $y_i \neq s$, ossia, ai nodi aventi distanza $d(s, y_i) = 2$. La fase di risoluzione di Saturation può quindi essere utilizzata per fornire agli agenti le informazioni necessarie: partendo dai nodi saturi, quando un'entità riceve l'informazione mancante da un suo vicino, necessaria al calcolo dell'eccentricità, può calcolare tale quantità di interesse ed inoltrare successivamente l'informazione mancante ai suoi vicini.**

Si osserva esplicitamente che, nella fase di risoluzione, un'entità invia informazioni diverse a ciascun suo vicino, al contrario delle fasi di risoluzione utilizzate in precedenza.

Il protocollo è il seguente:

```

if state == "PROCESSING":
    receiving("Resolution", dist)
    RESOLVE

Procedure INITIALIZE:
    Distance[x] = 0

Procedure PREPARE_MESSAGE:
    maxdist = 1 + MAX{Distance[*]}
    M = ("SATURATION", maxdist)

Procedure RESOLVE:
    PROCESS_MESSAGE
    CALCULATE_ECCENTRICITY
    for all y in N(x) - {parent}:
        maxdist = 1 + MAX{Distance[z] : z in N(x) - {parent, y}}
        send("Resolution", maxdist) to y
    state = "DONE"

Procedure PROCESS_MESSAGE:
    Distance[sender] = Received_distance

Procedure CALCULATE_ECCENTRICITY:
    r(x) = MAX{Distance[z] : z in N(x)}
```

Si osserva che, anche se ogni nodo riceve un messaggio distinto nella fase di risoluzione, un singolo messaggio verrà ricevuto da ogni nodo in tale fase, eccetto per i nodi saturi, che non ne riceveranno nessuno.

In termini di message complexity, quindi, il protocollo invia i messaggi della saturazione, più i messaggi per effettuare il calcolo delle eccentricità. In totale si ottiene

$$M[Eccentricity] = 3n + k - 4 \leq 4n - 4$$

dove k indica il numero di iniziatori.

Tale numero di messaggi coincide con il numero di messaggi nel protocollo utilizzato per il calcolo del minimo, quindi

$$T[Eccentricity] = T[Saturation] + \max\{d(s, x) : s \in Sat, x \in V\}$$

dove Sat è l'insieme dei nodi saturi.

Calcolo dei centri

Il centro di un grafo $G = (V, E)$ è il nodo avente eccentricità minima:

$$v \in V \text{ è il centro di } G \iff v = \arg \min_{x \in V} r(x)$$

Una rete può avere più di un centro.

Il problema del calcolo dei centri consiste nel far sì che ogni entità della rete sappia se sia o meno un centro, terminando quindi nei rispettivi stati finali center e not-center.

Protocollo 1

Una prima idea per risolvere il problema può essere quella di sfruttare i protocolli visti in precedenza, utilizzando proprio il fatto che un nodo è centro di una rete se e solo se è il nodo con eccentricità minima. Quindi, si può operare come segue:

1. Si esegue il protocollo per il calcolo delle eccentricità.
2. Si eseguono le ultime due fasi (saturazione e risoluzione) del protocollo per il calcolo del minimo.

La parte (1) viene iniziata dagli iniziatori.

La parte (2) inizierà dalle foglie una volta che, al termine della loro esecuzione del protocollo per il calcolo delle eccentricità, conoscono le loro eccentricità. La fase di saturazione del protocollo per il calcolo del minimo determinerà, nei due nuovi nodi saturi, il minimo tra tutte le eccentricità. Tale informazione verrà poi mandata in broadcast, partendo da essi, in tutta la rete. Quando un agente riceve il messaggio contenente l'eccentricità minima potrà poi determinare se questo è un centro o meno.

Questo approccio ha un costo di $3n + k - 4$ messaggi per la prima parte, dove k è il numero di iniziatori, ed $n + n - 2 = 2(n - 1)$ messaggi per la seconda parte, per un totale di $5n + k - 6 \leq 6n - 6$ messaggi.

Si vede un protocollo migliore

Protocollo 2

Si vuole sfruttare la struttura del problema con maggior dettaglio.

Si ricorda che $d[x, y] = \max\{d(x, z) : z \in T[y - x]\}$ è la massima distanza tra x ed i nodi in $T[y - x]$. Siano $d_1[x] = \max\{d[x, y] : y \in N(x)\}$ e $d_2[x] = \max\{d[x, y] : y \in N(x)\} \setminus \{d_1[x]\}$, ossia, rispettivamente, il primo ed il secondo elemento più grande di $\{d[x, y] : y \in N(x)\}$.

Il centri di un albero posseggono le seguenti proprietà:

Lemma 1: In un albero, o vi è un unico centro, o vi sono due centri i quali sono adiacenti tra loro.

Definizione: Sia $G = (V, E)$ un grafo. Siano $x, y \in V$ e sia

$d[x, y] = \max\{d(x, z) : z \in T[y - x]\}$. La più grande distanza tra due nodi di G prende il nome di diametro del grafo, e la si indica con $diam(G)$. Se $d[x, y] = diam(G)$, allora il cammino tra x ed y , sia questo $p(x, y) \subseteq E$, prende il nome di cammino diametrale.

Lemma 2: In un albero, tutti i centri appartengono a tutti i cammini diametrali.

Lemma 3: Un nodo x è un centro se e solo se $d_1[x] - d_2[x] \leq 1$. In particolar modo, se $d_1[x] - d_2[x] < 1$, allora x è l'unico centro.

Lemma 4: Siano y e z vicini di x tali per cui $d_1[x] = d[x, y]$ e $d_2[x] = d[x, z]$. Se $d[x, y] - d[x, z] > 1$, allora tutti i centri si trovano in $T[y - x]$.

Il lemma 3 fornisce un tool per la definizione di un protocollo risolutivo: un'entità x può determinare se sia un centro o meno, assumendo che conosca $d[x, y]$ per ogni suo vicino y . Ma questa è proprio l'esatta informazione fornita ad x dal protocollo per il calcolo delle eccentricità affinché potesse calcolare $r(x)$.

Ciò vuol dire che, per risolvere il problema del calcolo dei centri, è sufficiente eseguire il protocollo per il calcolo delle eccentricità. Quando un'entità ha tutte le informazioni per calcolare l'eccentricità, verificherà se il primo ed il secondo più grande valore ricevuto differiscono di al più uno. Se ciò è vero, entra nello stato di center, altrimenti entra nello stato di not center.

Con questo approccio, message e time complexity coincidono con quelle del protocollo per il calcolo delle eccentricità.

Un plug-in efficiente

Le soluzioni discusse fino ad ora sono *protocolli completi*. In certe circostanze, invece, un plug-in è sufficiente. Tali situazioni si verificano quando il problema da risolvere non consiste nel calcolo dei centri, ma questi ultimi devono iniziare una qualche computazione globale. In tali circostanze, l'obiettivo è solo quello di far modo che i centri sappiano effettivamente d'esserlo.

In un caso del genere, si può costruire un meccanismo più efficiente, sempre basato sulla

saturazione ma che utilizza la fase di risoluzione in una maniera differente.

Le proprietà espresse dai lemmi 3 e 4 permettono di progettare un protocollo plug-in efficiente.

Infatti, per il lemma 3, x può determinare se questo sia un centro o meno dopo aver ricevuto il valore $d[x, y]$ da ciascun suo vicino y . Ancor di più, se x non è un centro, per il lemma 4, tale informazione è sufficiente per determinare in quale sottoalbero $T[y - x]$ risiede un centro.

Quindi, la soluzione consiste nel fornire tali valori ad un nodo x , il quale dovrà determinare poi se è un centro o meno. Se non lo è, si avanza sino a quando non si incontra un centro. Per ottenere tali informazioni, si possono utilizzare le prime due fasi (quella di Wake-up e quella di Saturation) del protocollo per il calcolo delle eccentricità. Una volta che un nodo diventa saturo, può determinare se è un centro o meno controllando se il primo più grande ed il secondo più grande valore ricevuto differiscono al più di uno. Se non è un centro, il nodo saturo saprà che il centro (o i centri) dovranno trovarsi nella direzione dalla quale è stato ricevuto il più grande valore.

Mantenendo quindi traccia ad ogni nodo, durante la fase di saturazione, di quale vicino ha inviato il più grande valore, è possibile quindi determinare la direzione del centro. Ancor di più, un nodo saturo può decidere se si trova più vicino ad un centro o al padre di quest'ultimo. Il nodo saturo più vicino al centro, sia questo x , invierà poi un messaggio "Center", contenente il secondo più grande valore ricevuto incrementato di uno, in direzione del centro.

Un nodo nello stato di processing che riceve tale messaggio sarà, a sua volta, in grado di determinare se sia un centro o meno, e in quest'ultimo caso, la direzione verso i centri. Una volta che il messaggio giunge ad un centro c , c sarà in grado di determinare se è l'unico centro o meno, utilizzando il lemma 3. Nel caso ci fossero due centri, c è in grado di determinare quale sia il suo vicino che rispetta tale proprietà, e notificarlo.

Il plug-in *Center-finding* sarà quindi il plug-in *Saturation*, con l'aggiunta del messaggio "Center" che viaggia da i nodi saturi verso i centri

```
if state == "PROCESSING":
    receiving("Center". value)
    PROCESS_MESSAGE
    RESOLVE

Procedure INITIALIZE:
    Max_Value = 0
    Max2_Value = 0

Procedure PREPARE_MESSAGE:
    M = ("Saturation", Max_Value+1)

Procedure PROCESS_MESSAGE:
    if Max_Counter < Received_value:
        Max2_value = Max_Value
        Max_Value = Received_Value
```

```

        Max_Neighbor = seder
    else:
        if Max2_Value < Received_value:
            Max2_Value = Received_value

Procedure RESOLVE:
    if Max_Value - Max2_Value == 1:
        if Max_Neighbor != parent:
            send(Center, Max2_Value) to Max_Neighbor
            state = "CENTER"
    else:
        if Max_Value - Max2_Value > 1:
            send(Center, Max2_Value) to Max_Neighbor
        else:
            state = "CENTER"

```

Il costo dei messaggi di questo plug-in è determinabile osservando che, dopo l'applicazione del plug-in *Saturation*, un messaggio viaggerà dal nodo saturo s più vicino ad un centro verso il centro c dal quale è più distante. Quindi, **vengono scambiati ulteriori** $d(s, c)$ **messaggi. Essendo** $d(s, c) \leq n/2$, **il numero totale di scambio di messaggi ammonta ad:**

$$M[\text{Center-Finding}] = 2.5n + k - 2 \leq 3.5n - 2$$

dove k **è il numero degli initiator.**